

Business Entity Resolution Team Handbook

From private GitHub setup to a reproducible final submission.

Prepared for teammates joining a beginner-led ML team. Assumes basic Python and familiarity with linear or logistic regression; advanced ideas are introduced only when they become useful.

The objective	The working constraints
Maximize entity-level macro F0.5 . Aim for a strong private-leaderboard result; 0.99 is an aspiration, not a promise.	30 hours of core hands-on work, with up to 10 additional hours if useful.
Find every matching Source 2/3 record for each Source 1 business; return an empty list when appropriate.	Two machines with 16 GB system RAM; optional RTX 5060 laptop GPU. AWS GPU quota currently blocks the requested instance.
Learn, implement, measure, then improve.	Deadline in supplied guidelines: 27 September 2026, 23:59 IST .

What exists now

The project has a local Python environment, a guided notebook, streaming TSV readers, structural audits, documentation, and an experiment-log template. The scorer exercises, validation splits, retrieval, models, and predictions are not implemented.

A local Git remote points to **g0so/amazon-ml-challenge-2026**. Remote visibility, collaborator access, and latest pushed state have not been independently checked. The owner must confirm the Private badge before team use.

Read first

Everyone: pages 2-6. Repository owner: page 7. Teammates: pages 8-11. All implementers: Phase 1 on pages 12-14. Keep pages 22-24 available for the deadline.

Prepared 26 September 2026. Based on the supplied organizer documents, local file audits, and official technical documentation. Organizer clarifications override this handbook.

01 / TEAM KICKOFF

Start together, then divide the work

A 20-minute kickoff

1. Confirm access and ownership

The repository owner confirms it is private, invites teammates, and names one submission coordinator. Each person uses their own GitHub account.

2. Agree on shared definitions

Everyone reads the scoring examples on page 5. Confirm that an entity can have multiple matches and that a correctly empty prediction earns credit.

3. Choose task owners

Assign the roles below based on team size. These are responsibilities, not a requirement to have four people. One person may hold several roles.

Role	Owns	Hands off
Integration / submission	Working main branch, environment, releases, portal uploads	Versioned runnable package and submission log
Data / validation	Data audit, metric, group split manifest	Fixed scorer and common evaluation IDs
Retrieval / features	Candidate routes, recall ceiling, pair features	Candidate version, feature definitions, runtime
Models / analysis	Logistic baseline, thresholds, errors, controlled comparisons	Scores, predictions, and recommendation

The common contract

All experiments use the same scorer and saved split manifest. A candidate pair is one Source 1 ID and one Source 2/3 ID. Its label comes from training ground truth. Features must be available at inference. Keep IDs for joins, not as numeric similarity signals.

Coordinate work through small GitHub issues: hypothesis, owner, input version, output, acceptance check. Check in at phase boundaries or every 2-3 work hours; avoid frequent meetings that interrupt implementation.

Where to go next

Pages	Topic
3-6	Rules, data, scoring, hardware and schedule
7-11	Owner setup, teammate setup, Git workflow, project map
12-14	Phase 1: inspect, implement metric, design validation
15-18	Baseline, retrieval, logistic regression, error analysis
19-21	Validation discipline, optional models, experiment decisions
22-24	Submission, troubleshooting, glossary and sources

02 / OFFICIAL REQUIREMENTS

Rules that constrain every experiment

The task and submission rules below come from the supplied problem statement and guidelines [S1-S3]. Keep both documents available; do not resolve ambiguities by guessing.

Requirement	What the team must do
Time and submissions	Finish by 27 September 2026, 23:59 IST. Maximum five leaderboard submissions per day. Preserve every uploaded version.
Allowed data	Use provided data. No external business lookup, registration databases, commercial entity resolution, geocoding APIs, or internet data augmentation.
Model restriction	The final model must meet the stated MIT/Apache 2.0 license requirement and be no larger than 8 billion parameters. Verify the exact model and dependencies before adoption; clarify ambiguous licensing with organizers.
Leaderboard upload	Upload <code>matching_results.tsv</code> for the full test Source 1 set. Public/private splitting happens during scoring.
Final package	Both output TSVs, runnable source code, pinned environment, reproduction instructions, and methodology documentation are required.
Account conduct	No multiple-ID cheating or plagiarism. Each participant must avoid simultaneous challenge-portal logins. GitHub collaboration does not require sharing portal credentials.

Document conflicts: record them, then use a safe plan

- **Write-up length:** guidelines request 1-2 pages; task materials allow unlimited methodology detail. Prepare a concise 1-2-page summary plus the detailed template, and ask which files the final portal expects.
- **Ranking:** task statement says private results determine final ranking; guidelines mention both boards. Seek clarification. Select models using local evidence rather than chasing public fluctuations.
- **Unknown IDs:** task statement says rejection; validator comments mention score reduction. Always emit only valid test Source 2/3 IDs regardless.

Keep a rules log

Record organizer clarification, date, source, and the action it changes. This handbook does not grant permission to use external lookup data or unverified pretrained models.

03 / DATA UNDERSTANDING

What is in the dataset?

Source 1 is the deduplicated reference. A row in Source 2 or 3 is a noisy observation that may match a reference or be a distractor. The goal is identity matching, not predicting a price or selecting one class per business.

Split	Source 1	Source 2	Source 3
Training	2,206,821	5,034,616	5,285,603
Test	1,732,544	4,887,273	5,082,316

Counts are from the local full-file audit [S4], not estimates. Comparing all test references with all possible target records would require about 17.3 trillion comparisons. A retrieval stage must reduce that space.

Field / file	Interpretation
entity_id	Record key, including an S1-, S2-, or S3- prefix. Do not use numeric ID proximity or row order as evidence of identity.
business_name	May contain typos, abbreviations, reordered words, alternate names, or different scripts.
business_address	May be incomplete, reordered, abbreviated, or blank. Missing is different from conflicting.
country	Open set of string labels. Training: US and India. Test additionally includes France.
train_ground_truth.tsv	One reference ID and a comma-separated list of true matching target IDs; empty means no matches.

Facts that should influence experiments

- Training contains 7,638,365 positive links and 123,247 singleton references (about 5.6%). Observed true match counts range from 0 to 11; do not hard-code 11 as a universal limit.
- About 3.3% of training target addresses are blank. Names and addresses must provide complementary evidence.
- Training references are about 60% US / 40% India. Test is about 38.3% US / 46.8% India / 15.0% France. France has no labeled training examples.

Read every file as UTF-8 tab-separated text. Commas inside addresses are ordinary text; commas inside the match-list field separate IDs. Preserve original records and raw text.

04 / THE EVALUATION CONTRACT

Understand the score before modeling

For one reference entity, let T be the set of true target IDs and P the set of predicted target IDs. Count $TP = |T \cap P|$, $FP = |P - T|$, and $FN = |T - P|$.

Per-entity score

$F0.5 = 1.25 TP / (1.25 TP + FP + 0.25 FN)$.

If both sets are empty, score 1. If truth is empty but prediction is not, score 0.

True IDs	Predicted IDs	Score	Interpretation
A, B	A, B	1.000000	All links correct
A, B	A	0.833333	One missed link
A, B	A, B, C	0.714286	One false merge
A, B	empty	0.000000	All links missed
empty	empty	1.000000	Correct singleton
empty	C	0.000000	False merge on singleton

The final score is the arithmetic mean of these entity scores across **all** required references. Every reference has equal weight even if one has many more true matches. A missing prediction row is a format error, not a way to avoid a hard case.

Three common measurement mistakes

- Pair accuracy can look excellent when a model predicts nearly everything as a nonmatch. Most possible pairs really are nonmatches.
- Computing one F0.5 from pooled TP/FP/FN is a micro-style calculation. It is not the required entity macro average.
- A library option named `average="macro"` may average over classes. It does not automatically implement this competition metric.

What does the 99% ambition mean?

The reported public leader score of 98.4% is a user-supplied observation, not independently verified here. Moving from 0.984 to 0.990 would reduce average score shortfall by 37.5%. That requires substantial improvement in the remaining hard cases; it is not guaranteed.

An all-empty predictor scores $123247 / 2206821 =$ approximately 0.055848 on training. Use this as a scorer sanity check, not as a useful final model.

05 / RESOURCE PLAN

Hardware, time, and the AWS decision

Resource	Use now	Do not assume
Ubuntu i5 / 16 GB RAM	Primary development, streaming audit, CPU baselines, retrieval benchmarks	A full in-memory pair table will fit
Ryzen / 16 GB / RTX 5060	Optional independent experiment; later GPU work if justified	GPU VRAM equals system RAM, or two machines combine into 32 GB
AWS credits / 4 GB notebook	Optional small checks; keep credits for a measured later need	Credits grant instance quota or guarantee availability

Your error names **Studio JupyterLab Apps running on ml.g6.xlarge**: applied quota 0, usage 0, request 1. AWS lists a zero default for this GPU app type [S8]. It does not establish a universal 4 GB limit or prove all CPU instances are blocked.

Optional owner action: in the same Region, open Service Quotas, select SageMaker AI, find that exact app quota, and request 1 if adjustable. A notebook-instance or training-job quota is different. Approval can be delayed or denied [S9]. Continue locally while this is unresolved.

30-hour hands-on allocation

Phase	Hours	Exit checkpoint
1. Foundations	4	Verified scorer and validation design
2. Conservative baseline	3	Reproducible first result
3. Retrieval	7	Coverage, oracle score, RAM and time measured
4. Logistic regression	5	Learned matcher evaluated fairly
5. Errors and threshold	4	Targeted improvements confirmed
6. Boosted-tree comparison	3	Keep only if it improves reliably
7. Full run and package	4	Outputs and reproduction verified

This is hands-on time, not a compute-runtime guarantee. Allocate the optional 10 hours to the measured bottleneck. Benchmark full-run throughput in Phase 3 and reserve enough elapsed time for a rerun before the deadline. Drop optional complexity before dropping validation.

06 / GITHUB OWNER CHECKLIST

Repository owner: privacy and access

The local project already has origin pointing to `g0so/amazon-ml-challenge-2026`. Do not recreate it or reinitialize Git. The local remote alone does not verify that the remote repository is private or fully updated.

1. Verify privacy

Open [the team repository](#). Confirm the Private badge. If you cannot see it, resolve access or visibility before inviting people. With GitHub CLI, `gh repo view --json visibility` reports the current visibility.

2. Invite teammates

Collect their GitHub usernames. In repository Settings, open Collaborators, choose Add people, and send invitations. Each teammate must accept using their own account. Private-repository collaborators are supported on GitHub Free [S6].

3. Confirm the uploaded state

Check that `README.md`, `requirements.txt`, `project_config.json`, `src/`, `scripts/`, `notebooks/`, and `docs/` appear online. Never upload `.venv`, credentials, or raw datasets. The existing `.gitignore` excludes those normal locations.

4. Add this handbook when ready

The PDF is saved locally. Its creation does not push it to GitHub. Review and commit the documentation change, then push it to your chosen branch and use a pull request.

```
git status --short
git add .gitignore README.md output/pdf/team_handbook.pdf
git diff --cached --stat
git commit -m "Add team hackathon handbook"
git push
```

Run the commands on the branch you intend to share. If it is a new branch, use `git push -u origin BRANCH_NAME`. Do not force-push to main. Before staging, review any unrelated changes in your working tree.

Only if setting up a genuinely new repository later

```
git init -b main
git add .
git diff --cached --stat
git commit -m "Initialize team workspace"
gh repo create REPO_NAME --private --source=. --remote=origin --push
```

These are reference instructions, not steps to repeat on the existing repository. Authenticate first with `gh auth login` and a browser. [S5]

07 / LOCAL ONBOARDING

Teammate setup on Ubuntu

First accept the GitHub invitation and confirm you can open the private repository. Use your own account. These commands create a new clone; do not run git init inside a clone.

1. Install Git tools and sign in

Run the commands below. Choose GitHub.com, HTTPS, browser authentication, and allow Git authentication when prompted.

```
sudo apt update
sudo apt install git gh python3-venv
gh auth login
gh repo clone g0so/amazon-ml-challenge-2026
cd amazon-ml-challenge-2026
git config user.name "Your Name"
git config user.email "YOUR_GITHUB_EMAIL"
```

2. Create your own environment

Check python3 --version. The verified owner environment uses Python 3.13.12. Prefer a compatible Python 3.13 installation; package compatibility on another Python version must be checked. A virtual environment isolates project packages [S7].

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python -m ipykernel install --prefix .venv \
  --name amazon-ml --display-name "Amazon ML (local)"
```

3. Restore organizer data locally

Download/extract the organizer resource package using the authorized challenge access. Preserve the directory structure shown on page 11. Raw datasets are absent from Git by design.

4. Check and open

Run these from the cloned project root. Open the localhost link printed by Jupyter and select the Amazon ML (local) kernel.

```
.venv/bin/python scripts/check_environment.py
.venv/bin/python scripts/audit_data.py
bash start_notebook.sh
```

Before installing random fixes

If a dependency fails, share the Python version and first relevant error. Do not silently upgrade the shared requirements file. Resolve compatibility on a branch and record the resulting environment.

08 / POWERSHELL ONBOARDING

Teammate setup on Windows

The GPU laptop can use Windows for exploration. Do not copy the Ubuntu .venv: environments contain machine-specific paths and binaries. GPU drivers and deep-learning packages are not required for Phase 1.

1. Install prerequisites

Install Git for Windows, GitHub CLI, and Python from their official distributions [S10]. Prefer Python 3.13 for the pinned environment. Reopen PowerShell after installation, then check `git --version`, `gh --version`, and `python --version`.

```
gh auth login
gh repo clone g0so/amazon-ml-challenge-2026
cd amazon-ml-challenge-2026
git config user.name "Your Name"
git config user.email "YOUR_GITHUB_EMAIL"
```

2. Create and install

Use the environment executable directly, so no PowerShell activation-policy change is necessary.

```
python -3.13 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m ipykernel install --prefix .venv --name amazon-ml
```

3. Add the local dataset and verify

Extract the same organizer package layout as on Ubuntu. Then run:

```
.\.venv\Scripts\python.exe scripts/check_environment.py
.\.venv\Scripts\python.exe scripts/audit_data.py
.\.venv\Scripts\python.exe -m jupyterlab
```

In the browser, open `notebooks/01_phase1_foundations.ipynb` and choose the `amazon-ml` kernel. `start_notebook.sh` is a Linux launcher; do not run it as a PowerShell command.

If the Linux-pinned environment fails on Windows

Stop and record the package and error. A platform-specific dependency may need a separate environment file. With the team, resolve the direct dependencies from `requirements.in` into a clean Windows environment, run the setup checks, and save a separate `requirements-windows.txt`. Do not overwrite the tested Linux lock file or claim the environments are identical.

GPU later

Confirm actual GPU VRAM and supported drivers before selecting any neural model. The GPU does not increase the 16 GB system RAM available for Python tables. Use the same split and feature definitions on both machines.

09 / TEAM COLLABORATION

Daily Git workflow without collisions

A commit is a saved version. A branch isolates a task. A push uploads commits. A pull updates your local branch. A pull request (PR) asks teammates to review and merge a branch. Prefer one small task per branch.

1. Start from an updated main

Ensure git status shows no uncommitted work. If it does, finish or commit it before switching. Replace `alice-metric` with a descriptive unique branch name.

```
git switch main
git pull --ff-only
git switch -c alice-metric
```

2. Implement one agreed task

Use the issue acceptance checks. Keep reusable logic in `src/` and exploration in your own notebook. Ask before changing shared scorer or split definitions.

3. Review, commit, push

Stage actual changed files deliberately. The example path below is only illustrative; use the files you edited.

```
git status --short
git diff
git add src/business_entity_resolution/metrics.py
git diff --cached
git commit -m "Implement entity-level metric with edge-case checks"
git push -u origin alice-metric
```

4. Open and review a PR

On GitHub, compare the branch with main. State the problem, change, checks, result, and limitations. Another teammate reviews, requests fixes if needed, and merges. Automated branch protection is optional; agree on the practice even if unavailable.

When main changes during your work

Commit your changes, then run `git fetch origin` and `git merge origin/main` on your task branch. If conflicts appear, inspect each file and agree on the intended combined behavior. Remove conflict markers, rerun relevant checks, stage resolved files, and complete the merge commit. Avoid destructive resets or force-pushes as a shortcut.

Notebooks need ownership

Do not edit the same notebook simultaneously. Use separate notebooks such as `02_alice_baseline.ipynb` and `02_bob_retrieval.ipynb`. Clear bulky outputs before committing; preserve numerical results in reports. Share reusable functions through reviewed Python modules.

10 / PROJECT MAP AND DATA TRANSFER

Know where every file belongs

Location	Purpose and team policy
README.md	Entry point; keep setup steps current.
docs/	Roadmap, Phase 1 instructions, and AWS diagnosis.
notebooks/	Guided exercises and person-owned exploration.
src/business_entity_resolution/	Reusable readers now; shared scoring/features/models later.
scripts/	Commands for environment and streaming audits; future pipeline entry points.
project_config.json	Resource directory, seed, and conservative defaults. Four-GiB target is guidance, not an enforced cap.
requirements.in / requirements.txt	Direct package needs / pinned verified environment. Do not edit pins casually.
reports/	Audit outputs and experiment log; never label preview statistics as full-data statistics.
data/processed/	Future derived data and saved split manifests. Mostly Git-ignored; arrange explicit versioned sharing of required manifests.
artifacts/ and output/	Model/cache files and final TSVs; ignored by default. Only the handbook PDF is explicitly allowed for Git.

Expected local dataset layout

```

6ab10eb3b23ba_student_resource/
  student_resource/
    dataset/
      train/
        train_source1.tsv
        train_source2.tsv
        train_source3.tsv
        train_ground_truth.tsv
      test/
        test_source1.tsv
        test_source2.tsv
        test_source3.tsv

```

If your extraction has an extra enclosing folder, move the extracted package to the expected location or edit `resource_dir` deliberately. The helper resolves paths from the project root. Do not edit raw TSVs. Compare the audit counts after transfer; for exact reproducibility, also record file checksums when practical.

Git is for code and small documentation here. Use organizer access or an agreed private team transfer for large data/artifacts. A clone cannot recreate excluded datasets by itself. Do not send AWS keys, tokens, or a copy of a virtual environment.

11 / FIRST FOUR HOURS

Phase 1A: environment, files, and labels

Open notebooks/01_phase1_foundations.ipynb. Work in order. The early cells run; unfinished exercise checks are intentionally disabled. A skipped exercise is not a passed test.

1. Inspect resources - 15 minutes

Run the environment cells. Confirm the interpreter belongs to .venv, all seven inputs exist, and enough RAM is available. Start with one kernel and two threads. Do not load all files into dataframes.

2. Inspect schemas - 20 minutes

Run the five-row previews. Verify four columns in source files and two in ground truth. Explain why a tab separator is needed. Check that blank addresses remain blank and Unicode text is preserved.

3. Parse match lists - 30 minutes

Implement `parse_match_ids` yourself. A blank string becomes an empty set; a nonblank string is a list of IDs separated by commas. Check malformed and repeated IDs before converting to a set.

Input	Expected interpretation
empty string	Zero matches, not one empty-string ID
S2-123,S3-456	Two target IDs
S2-123,S2-123	Duplicate-label issue to detect
S1-123	Invalid target-source prefix
S2-123,,S3-456	Malformed empty token to detect

Audit checkpoints

The full supplied data has 2,206,821 truth rows; 123,247 empty lists; 7,638,365 positive links; observed match counts 0-11. `reports/data_audit_full.md` contains the structural audit. Preview reports scan file heads and are not representative samples.

What the current audit does not prove

It checks structure and counts, including repeated IDs inside each truth list. It does not prove global ID uniqueness, valid cross-file references, one business per label group, or label correctness. These remain data-integrity tasks. Use chunking or disk-backed checks instead of giant dictionaries if memory is tight.

Stop-and-fix conditions

One giant column means a separator problem. A literal "nan" token signals missing-value mishandling. A restarted kernel suggests memory pressure. Resolve these before computing model scores.

12 / SCORER AND LABELED EXAMPLES

Phase 1B: implement your measurement

4. Implement score_entity - 45 minutes

Use the set-count formula on page 5. Handle the both-empty case explicitly. Implement the function rather than copying a generic classification metric that averages the wrong quantity.

Enable run_score_checks in the notebook only after the function works. All six cases on page 5 must pass. Then write your macro scorer: iterate over every required reference and average its score. Assert that prediction keys cover those references exactly.

Two aggregate checks

- Use the last three rows of the scoring table as a three-entity dataset. Scores are 0, 1, 0, so the average must be 1/3.
- Predict an empty set for every training reference. The result must be approximately 0.055848. If it is 1, 0, or close to 1, inspect singleton handling and the averaging denominator.

5. Inspect 20-30 complete groups - 40 minutes

Select references from both countries, with zero, one, and several matches. Include missing target addresses and different scripts where available. Find the target IDs in ground truth, then stream S2/S3 to collect just those records.

Do not build a Python dictionary of every target solely to inspect 30 examples. Keep a small set of requested IDs, scan each target file once, and retain only needed rows. This is a data-access exercise, not a matching algorithm.

What to record	Why it matters
Name change	Abbreviations, legal suffixes, typos, order, aliases, script changes
Address change	Missing components, reordering, shortened street names, conflicting numbers
Evidence preserved	Rare tokens, house number, locality, name fragments
Ambiguity	Same brand at another location, generic name, sparse fields
Possible future feature	A numerical measurement that captures the useful evidence

Human inspection boundary

Use training labels to understand errors. Do not infer test truth from external business lookup, manually patch test predictions using online identities, or treat a plausible guess as a verified label.

Phase 1B handoff

Commit your scorer, its edge-case checks, and a short explanation of the aggregation. Share five instructive training examples and the failure modes they suggest. The reviewer should be able to reproduce the checks before using the scorer for any model comparison.

13 / VALIDATION DESIGN

Phase 1C: build an honest split

6. Design before implementing - 45 minutes

The split unit is an underlying labeled business group: one Source 1 reference plus all its true target variants. Randomly splitting pairs can put the same business in training and evaluation.

Partition	Starting share	Purpose
Training	70% of groups	Fit models and learned preprocessing
Development	15% of groups	Features, model choices, threshold tuning
Locked holdout	15% of groups	Confirm the chosen approach after development

These are proposed proportions, not existing split files. Preserve country and match-count buckets such as 0, 1, 2-4, 5+. Use a fixed seed and save the assignment manifest. Start with a manageable subset of complete training groups for development; do not sample each source independently.

Implementation checklist

- Validate reference uniqueness and target references. If a target belongs to multiple labeled references, investigate and keep the connected group together across the split.
- Keep every selected reference's true target variants in its partition. Otherwise you may falsely turn a matching business into a singleton.
- Assign unmatched target records separately as distractors. Do not invent a reference identity for them. Detect duplicates or suspicious overlap that could leak identity.
- Prevent held-out records from entering supervised training pairs, including negatives. Fit learned preprocessing and feature selection on the training side.
- Save seed, input versions, group definition, country counts, singleton counts, and overlap-check results. Give the manifest a version used by every experiment.

Why a small evaluation can look too good

A tiny candidate pool contains fewer confusing businesses. Evaluate with realistic distractors, and later stress-test against a larger pool without training on held-out identities. Score every held-out reference, including singletons and retrieval misses.

Phase 1 exit review

All scorer checks pass; file reading is correct; 20-30 groups inspected; split design and overlap risks documented. Do not claim validation exists until the manifest and checks are implemented. Learning-preprocessing leakage guidance: [S11].

14 / FIRST END-TO-END RESULT

Phase 2: a baseline you can explain

Budget: about 3 hands-on hours. The goal is a working, reproducible pipeline with a measurable score. It does not need advanced ML.

1. Preserve raw and normalized views

Keep original name and address strings. Add conservative normalized versions: consistent casing, whitespace, and selected punctuation handling. Validate transformations against training examples before expanding them.

2. Use a strict matching rule

Try country plus normalized name and normalized address equality as a conservative baseline. Treat blank fields explicitly. Several targets may match a reference. Do not force exactly one match or assume equality always proves identity.

3. Represent empty predictions explicitly

Every required reference needs a prediction set, including references with no candidates and references whose candidates are all rejected. Keep retrieval output separate from final decisions.

4. Evaluate with the shared scorer

Report macro F0.5, country scores, singleton false-positive rate, and runtime. Compare with the all-empty sanity baseline. Save the prediction version for error inspection.

Cleaning mistakes to avoid

Tempting change	Why it can fail	Better first experiment
Remove all numbers	Erases house, unit, or postal distinctions	Keep numbers and compare agreement/conflict
Strip all non-Latin text	Destroys names in other scripts	Preserve Unicode; add alternate representations later
Remove all common words	May merge distinct legal/brand names	Retain raw features; test a second normalized view
Treat missing as unequal	Rejects valid links with absent addresses	Use a separate missingness indicator

Deliverable

A script or notebook that reads the fixed development inputs, produces match sets, evaluates them, and saves a concise result. A teammate should rerun it without editing hidden notebook state. Do not tune rules on the locked holdout.

Decision rule

If the baseline is weak, first classify why: no candidate, rejected true match, accepted wrong match, or data/scoring bug. The next phase addresses candidate coverage; adding a classifier cannot recover a pair that never reaches it.

15 / THE MAIN SCALING CHALLENGE

Phase 3: retrieve plausible candidates

Budget: about 7 hours. Candidate generation (blocking) reduces trillions of possible comparisons to a manageable set. Retrieval tries to keep true matches; the matcher later rejects false ones.

Build complementary routes

- **Name route:** normalized exact names, informative shared tokens, then character-based similarity for small spelling changes.
- **Address route:** shared locality/number fragments plus partial name evidence. It can recover a badly damaged name.
- **Union:** combine and deduplicate results from different routes. Do not require every route to agree.

Only enforce country as a hard boundary after checking training positives support it. Handle any country label, including France. Do not use US/India-only filtering. A retrieval top-K is a compute choice, not a required number of final matches.

Learn the basic similarity ideas

Token overlap compares shared words. **Character n-grams** compare short overlapping character sequences, so one typo does not destroy all evidence. **TF-IDF** downweights common terms and highlights distinctive ones. **Cosine similarity** compares the direction of the resulting vectors [S12]. Different scripts may still need a different representation.

Measure	What it tells you
Candidate recall	Fraction of all true links retained; report by country and difficult slices too
Complete-entity coverage	Fraction of non-singletons with every true match retained
Oracle macro F0.5	Score if a perfect matcher chose only the true links inside each candidate set
Candidate counts / cost	Median, tail, maximum, total pairs, runtime, and peak RAM

The oracle is a score ceiling for that candidate set, including correctly empty singletons. If it is below your target, fix retrieval first. Measure at the last filtering stage before model inference, not just an early broad pass.

16 GB implementation discipline

Use indexed, sparse, batched, or disk-backed retrieval. Do not materialize an all-pairs matrix or convert a large sparse matrix to dense. Grow benchmarks gradually; record pair throughput and realistic full-test estimates.

16 / LOGISTIC REGRESSION MATCHER

Phase 4: use the model you already know

Budget: about 5 hours. One candidate pair becomes one feature vector x . Its label is 1 if the target is listed for the reference in training ground truth, and 0 otherwise, assuming the supplied labels are complete and integrity checks pass.

Connection to your background

$p(\text{same business} | x) = \text{sigmoid}(w \cdot x + b)$.

You know the logistic model; the important new skill is constructing evidence about a pair.

Feature	Reason to try it
Name token / character similarity	Word reordering and local spelling changes
Address token / character similarity	Location evidence independent of the exact name
Rare-token agreement	A distinctive token can be more useful than common words
House/unit/postal number conflict	High overall similarity can hide an important contradiction
Missing-field indicators	Distinguish absent evidence from disagreeing evidence
Name similarity x address similarity	An explicit interaction lets a linear model combine evidence

Train on realistic negatives

A random pair of unrelated businesses is usually easy. Include nonmatching pairs that the candidate generator actually retrieves: similar names, nearby addresses, or shared generic tokens. Mine harder negatives only from training data. Do not use held-out errors as training examples unless you explicitly create a new evaluation split.

You may sample negatives for tractable training. Record the sampling rule. Sampling changes apparent class balance, so probabilities may be miscalibrated. Evaluate on the real candidate distribution and choose a threshold using entity-level macro F0.5, not a default assumption that 0.5 is optimal.

Keep the comparison fair

- Freeze candidate and split versions when comparing features or model settings. Fit scalars and other learned transforms only on the training partition.
- Start with a small regularization comparison. Explain each feature, inspect suspicious coefficients, and verify that no feature derives from labels or entity-ID numbering.
- Save model settings, feature order, preprocessing, environment, and runtime. Confirm a fresh process reproduces the same predictions.

17 / ERROR ANALYSIS AND THRESHOLDS

Phase 5: improve the actual failures

Budget: about 4 hours. Save per-reference predictions and scores so the team can inspect mistakes. Sample errors systematically rather than choosing only dramatic examples.

Failure category	First investigation	Likely experiment
True link not retrieved	Which retrieval route missed it?	Add a complementary route or adjust candidate budget
Retrieved link rejected	Missing/contradictory features?	Improve features or decision threshold
Wrong link accepted	Similar brand, different location?	Number-conflict or distinctive-token features
Singleton false merge	Weak top candidate accepted?	More conservative decisions for supported cases
Script / missing-address failure	Representation lacks useful evidence?	Alternative text view; explicit missingness
Ambiguous training truth	Conflicting evidence or labeling issue?	Document; do not silently rewrite truth

Threshold optimization step by step

1. Score development candidates

Generate continuous model scores for every candidate using the same retrieval conditions as evaluation.

2. Sweep a modest threshold grid

At each threshold, rebuild predicted ID sets for all references and compute macro F0.5. Include empty lists. Record singleton and country effects.

3. Look for stable gains

Prefer a reasonably stable threshold region over a tiny isolated peak. Keep the locked holdout out of the sweep. Probability calibration, if needed, uses separate or out-of-fold data.

4. Confirm the chosen decision rule

Only add source- or missingness-specific thresholds if repeated evidence supports them. France has no labels for directly tuning a France threshold.

Track effect, not just intuition

For each change, predict which error category should improve, then measure it. Run an ablation: remove the new component and see whether the gain disappears. If two models disagree, inspect those disagreements before combining them.

What to bring to a coaching review

Baseline and new score; split/candidate versions; country and singleton breakdown; runtime/RAM; five representative errors; your explanation; the next hypothesis. A leaderboard number alone is not enough to diagnose progress.

18 / LEAKAGE AND LEADERBOARD CONTROL

Validation that remains believable

Keep these boundaries explicit

- **Identity boundary:** the same underlying business should not appear in supervised training and held-out evaluation through different record variants. Audit groups before constructing pairs.
- **Learning boundary:** fit scalers, vocabulary statistics, feature selection, and model parameters on the intended training partition. Deterministic string cleanup is different from learning from held-out data [S11].
- **Decision boundary:** choose features and thresholds on development data; use the locked holdout for confirmation after the approach is selected. Repeatedly adapting to it makes it another development set.
- **Retrieval boundary:** never insert missing ground-truth links into validation candidates to make the matcher look better. Evaluate the complete pipeline, including retrieval failures.

Country transfer and stress tests

Always show separate US and India results. Their proportions shift between train and test. Try training on US and evaluating India, then reversing the direction, to test dependence on country-specific patterns. These are stress tests, not measurements of France performance. Keep a country-agnostic fallback.

France represents about 15% of test references. A four-percentage-point gain on that country would change the full-test macro average by about 0.6 percentage points if other countries stayed constant. Private-subset proportions are unknown; do not assume a public gain transfers exactly.

When to use cross-validation

One fixed split is enough for early iteration. For serious finalists, use three business-group folds if time permits. Each fold repeats preprocessing, model fitting, and evaluation without identity overlap. Pool out-of-fold predictions for later analysis while preserving the intended tuning/confirmation separation.

Judge small improvements carefully

Compare candidates on the same entities. Inspect the distribution of per-entity score differences; bootstrap business groups or compare fold variation to assess stability when needed. Do not treat a tiny one-split increase as certain superiority.

Public leaderboard policy

Use the five daily submissions to confirm meaningful local improvements, not to reverse-engineer hidden labels. Log every upload and its code/data versions. Final selection should balance strong validation, transfer behavior, reproducibility, and the organizer ranking rules.

19 / PHASE 6 AND STRETCH WORK

Optional models must earn their place

Budget: about 3 hours for one boosted-tree comparison. Use the same candidate pairs, split, and features as logistic regression so the comparison isolates the model.

Why trees might help

A decision tree learns conditional rules. Gradient boosting adds trees sequentially to improve predictions, capturing interactions such as strong name agreement being useful only when address numbers do not contradict it. This can reduce the need to handcraft every interaction [S13].

1. Choose a compliant implementation

Verify the actual implementation/model licensing against the MIT/Apache 2.0 requirement. Record version and configuration. Do not infer license compliance from a generic model-family name.

2. Tune a few influential choices

Try modest tree depth or leaf count, minimum leaf size, learning rate, and regularization. Use early stopping on the designated development data where supported. Avoid a large exhaustive grid.

3. Retune the threshold fairly

Different models produce differently scaled scores. Give each the same threshold-selection protocol, then confirm the chosen configuration separately.

4. Keep only meaningful improvement

Compare macro score, error categories, memory, inference time, and country transfer. A small gain may not justify a full-run deadline risk.

Ensembling and stacking: later, if justified

Averaging compatible scores can help if models make different errors; retune the threshold. Simply unioning positive matches can damage precision. Stacking trains another model on base-model outputs and requires out-of-fold predictions for its training inputs. In-sample base predictions would give misleadingly optimistic features.

Neural matching: a targeted option

Consider a small compliant model only if analysis shows a representation problem that simpler features cannot address, and if GPU memory and runtime permit it. Retrieval is still needed at this scale. Confirm permitted use and the exact weights license; external business lookup remains prohibited.

Time-pressure decision

If Phase 6 threatens final inference or packaging, skip it. A tested simple pipeline with correct retrieval and scoring is a complete entry; an unfinished complex model is not.

20 / EVIDENCE AND HANDOFFS

Run experiments as a team

Before each run, write one sentence

Example: "Adding address-number conflict features should reduce false merges among similarly named businesses without materially hurting missing-address cases." This is a testable hypothesis; "try more models" is not.

Record	Minimum information
Identity	Experiment ID, owner, timestamp, branch/commit
Inputs	Data version, split version, candidate version, feature version
Configuration	Model settings, seed, sampling rule, threshold-selection protocol
Quality	Macro F0.5, country scores, singleton false-positive rate
Retrieval	Candidate recall, complete-entity coverage, oracle macro score
Resources	Candidate count, runtime, peak RAM, extrapolated full-run time
Decision	Keep/reject/uncertain, observed failure modes, next hypothesis

reports/experiments.csv already provides a starting schema. Add version fields deliberately if needed. CSV conflicts are still possible: assign one log editor or write per-experiment reports and consolidate at review time.

Use these questions to choose the next run

Observation	Next action
Oracle score below target	Improve retrieval; do not spend time tuning the classifier ceiling
High oracle, many false merges	Improve contradictory evidence and realistic negatives; inspect thresholds
High oracle, many rejected true links	Inspect representation, missingness, and overly conservative decisions
US good, India weak	Inspect script/address slices; avoid hiding failure in the overall average
Only public score improves	Investigate mismatch; do not assume generalizable progress
Quality improves but runtime explodes	Measure full-run feasibility; simplify or batch before accepting

Handoff format

Send your teammate: exact commit, command to run, required input versions, output location, measured result, and known limitation. A notebook that only works after an undocumented sequence of cells is not a reproducible handoff. Restart and run the relevant work from a clean state.

21 / FINAL SUBMISSION

Phase 7: make the result reproducible

Reserve at least four hands-on hours plus full inference time. Freeze the chosen configuration, rerun necessary preparation, then generate both outputs from the same inference run.

File	Exact columns / meaning
matching_results.tsv	source1_entity_id TAB matched_entity_ids. Final comma-separated S2/S3 IDs, or an empty field. This is the leaderboard file.
candidate_pairs.tsv	source1_entity_id TAB candidate_entity_ids. The exact final candidate set entering model inference, before match decisions. Required in the final package.

- Each file contains exactly one row per test Source 1 ID, including France and predicted singletons. No duplicate reference rows or duplicate IDs inside lists.
- Use tabs between columns and commas between IDs; emit only existing test S2/S3 IDs. Every final match must be present in its candidate list.
- Validate membership and subset rules yourself as needed: the supplied validator skips ID-existence checks by default, and candidate/subset issues can be warnings. A PASS is not proof of all semantic requirements.

From the original student_resource directory, use this template. Replace the paths with your actual absolute output paths:

```
python3 utils/validate_submission.py \
  --matching "/ABS/PATH/output/matching_results.tsv" \
  --candidate "/ABS/PATH/output/candidate_pairs.tsv" \
  --test-dir dataset/test --check-ids
```

The ID check can use considerable RAM. If necessary, validate in stages and perform a separate disk-backed candidate/subset integrity check. Do not silently skip a requirement because one helper is memory-heavy.

```
<team_name>_submission.zip
output/
  matching_results.tsv
  candidate_pairs.tsv
code/business_entity_resolution/
  src/
  README.md
  requirements.txt
Documentation_template.md
```

Fill the organizer methodology template; prepare the short summary requested by guidelines. Include model artifacts or reproducible training instructions as necessary. Test regeneration from the supplied data in a clean environment. Do not include .venv, raw data, or this handbook as a replacement for the required methodology.

The coordinator uploads matching_results.tsv, confirms SCORED status, records score/time/file checksum/commit, and retains the exact ZIP and output versions. Verify portal instructions for selecting the final solution rather than assuming the newest upload is automatically chosen.

22 / WHEN SOMETHING GOES WRONG

Troubleshooting and deadline checklist

Symptom	Check first	Action
GitHub repository not found	Invitation, signed-in account, exact URL	Accept invitation; use gh auth status; confirm owner granted access
origin already exists	Existing repository configuration	Inspect git remote -v; do not recreate or overwrite blindly
Push rejected	Remote updates or branch permissions	Fetch, merge/review changes, then push; do not force main
Import error in notebook	Kernel interpreter	Select project .venv; inspect sys.executable
Pinned package will not install	Python version, OS, failing package	Reproduce error; agree on a separate compatible environment
Dataset missing after clone	Raw data excluded from Git	Extract organizer data into the expected layout
Kernel dies / system swaps	Available memory and batch size	Stop oversized work; restart; shrink batches; avoid dense matrices
Candidate counts explode	Common tokens / broad blocking	Measure tail sizes; improve indexing and filter without hiding recall loss
High local score, low public score	Leakage, scoring, distribution shift	Audit end-to-end validation and output integrity
AWS quota error	Exact service/app/type and Region	Request correct quota if useful; continue locally

Before the final upload

- Chosen configuration and code version frozen; output files regenerated together; full-test reference coverage confirmed.
- No unknown target IDs, duplicate lists, missing France records, or final matches outside candidate sets.
- Pinned dependencies, entry commands, required artifacts, and filled methodology packaged. A teammate has reproduced the selected result or the complete small-scale workflow.
- Submission coordinator tracks daily limit and confirms acceptance/status. Preserve a known-good earlier version if the last experiment fails.

If behind schedule

Keep the best validated pipeline. Stop broad model searches. Protect time for complete inference, output checks, and submission. A small unconfirmed gain is not worth losing a valid entry.

23 / KEEP THIS PAGE HANDY

Glossary, references, and next action

Term	Meaning in this project
Entity resolution	Deciding which records refer to the same real business
Blocking / retrieval	Selecting plausible target records before expensive scoring
Hard negative	A different business that looks plausibly similar
Feature	A numerical measure of evidence about a candidate pair
Threshold	Score cutoff used to accept a candidate as a match
Leakage	Training or selection uses information that should remain held out
Out-of-fold prediction	Prediction made by a model that did not train on that business group
Oracle score	Best score possible if true links inside candidates were selected perfectly
Ablation	Remove one component to measure whether it caused improvement

Sources and authority

- [S1] Organizer problem statement: [6ab5628d5a817_amazon_ml_challenge_problem_statement.pdf](#).
 [S2] Organizer guidelines: [6ab56657b4f1a_guidelines_and_key_instructions_amazon_ml_challenge_2026.pdf](#).
 [S3] Organizer README.md, Documentation_template.md, and utils/validate_submission.py inside student_resource.
 [S4] Local reports/data_audit_full.json and reports/setup_verification.md. Counts are verified; global identity integrity is not.
 [S5] [GitHub: import local code](#)
 [S6] [GitHub: invite collaborators](#)
 [S7] [Python: virtual environments](#)
 [S8] [AWS: SageMaker quotas](#)
 [S9] [AWS: quota increase requests](#)
 [S10] [Official downloads: Git / GitHub CLI / Python](#)
 [S11] [scikit-learn: leakage and common pitfalls](#)
 [S12] [scikit-learn: text feature extraction](#)
 [S13] [scikit-learn: gradient boosting and ensembles](#)

Additional [S10] links: [GitHub CLI](#) | [Python downloads](#). Online guidance reviewed during this conversation; service behavior and availability can change.

Team next action

Owner verifies privacy and invitations. Each teammate clones, installs, and checks the data layout. Then everyone completes the Phase 1 scoring exercise; the validation owner consolidates the shared scorer and split design before Phase 2.